Домашнее задание №3. Аналитический пайплайн на объектном хранилище: собственный MapReduce и DuckDB

Автор: Шайкин Андрей Сергеевич. ИСУ: 468044. Группа ИИИ-3-ИнжДан-1-3

Вариант 2 — «Программист, разработчик на Python»

Репозиторий: https://github.com/Kekendr/hh-skills-mapreduce

Итоги работы

Этап	Что сделано	Результат
1. Хранилище	RustFS 1.0.1 на четырёх папках-дисках, схема EC:2	бакет `hh-lake`
2. Загрузка	`upload.py`, `export_segment.py`	847 619 384 байт, хеш сверен; сегмент варианта — 7 545 вакансий
3. MapReduce	26 сплитов по 32 МиБ через S3 Range, 6 мапперов, 4 редьюсера	3 052 навыка, 50 665 пар, 356 245 троек
3. DuckDB	SQL к тому же объекту через httpfs	те же числа
4. Сверка	`verify.py` по файлам из бакета	409 962 сочетания, 0 расхождений
5. Зарплаты	медианы, прибавки от пар и троек	повышают airflow, dwh, clickhouse, etl, asyncio, kafka; понижают html, css, unity, обучение
6. Регионы	Москва, Санкт-Петербург, Екатеринбург	медиана 191 400 / 150 000 / 110 000 ₽
Профилирование	5 запусков на конфигурацию, время и пик памяти	DuckDB быстрее MapReduce в 2,6 раза
Бонус	три оптимизации MapReduce, Parquet для DuckDB	MapReduce: память ×2,3; DuckDB: время ×2,8, память ×7,2

1. Стенд и хранилище

Компонент	Параметры
Объектное хранилище	RustFS 1.0.1, четыре папки-диска `D:\rustfs\data1…data4`, схема кодирования EC:2 (2 фрагмента данных и 2 чётности), API `127.0.0.1:9000`
Программы	Python 3.11.9, boto3 1.40.69, DuckDB 1.5.6, psutil 7.2.2, matplotlib 3.11.1
Машина	Windows 10, 6 физических / 12 логических ядер, 31,8 ГиБ памяти

Хранилище развёрнуто так же, как в ДЗ-2. Все четыре папки лежат на одном физическом диске, поэтому сервер запускается с переменной `RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true`. Бакет `hh-lake` разделён на зоны:

Префикс	Содержимое
`raw/it_vacancies_2024.csv`	исходный датасет, все сегменты
`segment/variant2_python_developers.csv`	вакансии варианта 2
`curated/it_vacancies_2024.parquet`	колоночная копия датасета (бонус)
`results/mapreduce/`, `results/duckdb/`	результаты подсчёта для k = 1, 2, 3 и сводки
`results/mapreduce_failtest/`	результат прогона с отказом маппера
`results/analysis/`, `results/profiling/`	таблицы этапов 5–6 и замеры

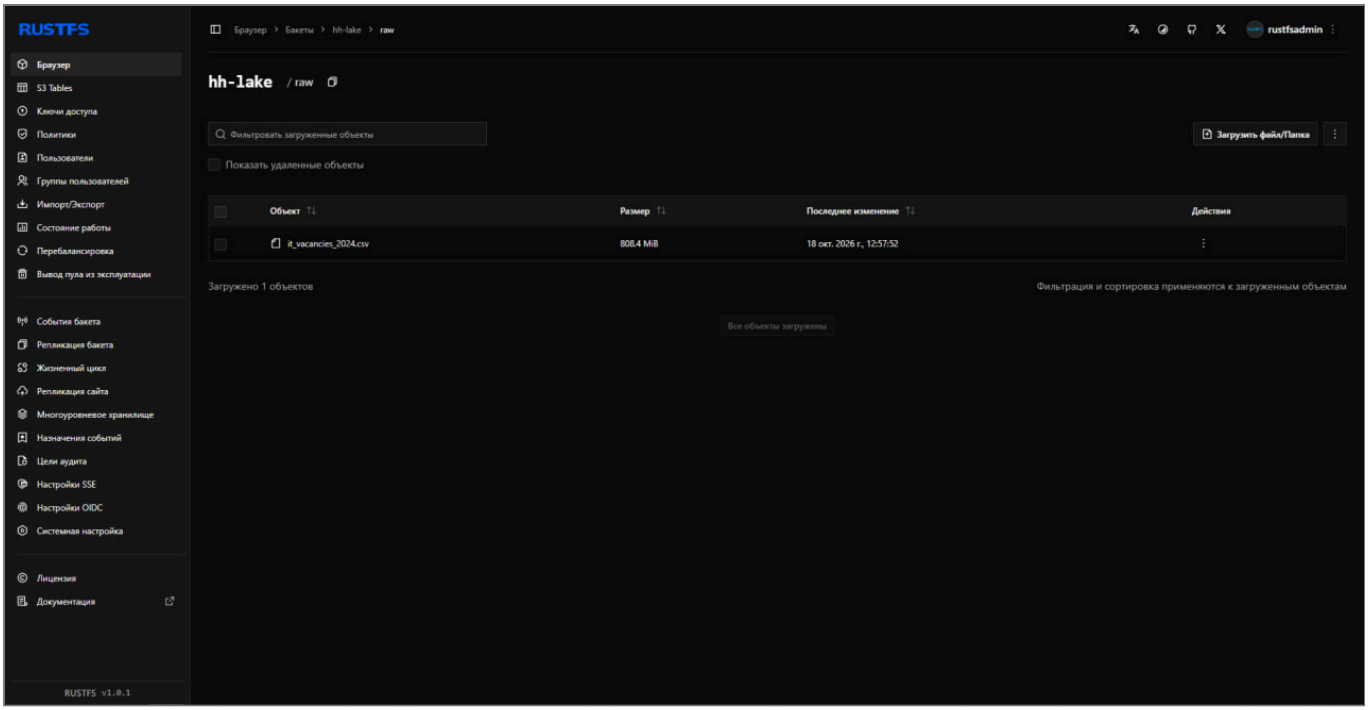


Рис. 1. Консоль RustFS: объект `raw/it_vacancies_2024.csv` в бакете hh-lake, 808,4 МиБ

2. Данные и загрузка в хранилище

Датасет `it_vacancies_2024.csv` — 229 388 вакансий hh.ru, 23 колонки, снимки с 23.08 по 15.09.2024, повторов `vacancy_id` нет. Колонка `role` содержит ровно те сегменты, что в списке вариантов: «Программист, разработчик» (87 803), «Аналитик» (30 112), «DevOps-инженер» (5 158) и другие. Архив `hh.zip` с 10 000 JSON — это подмножество тех же вакансий в формате API hh.ru, в работе он не нужен.

До написания кода проверены два свойства данных, от которых зависит корректность обоих способов подсчёта:

- физических строк в файле 229 389 — это ровно 229 388 записей и заголовок. Многострочных полей нет, поэтому файл можно резать на части по символу перевода строки, и ни одна вакансия не разрежется;
- у 1 499 навыков встречается разное написание регистра (`Docker`, `DOCKER`, `docker`). Без приведения к нижнему регистру один навык считался бы как несколько разных.

Загрузка выполнена скриптом `upload.py` через boto3 многочастной загрузкой по 64 МиБ — отсюда суффикс `-13` в ETag. Перед загрузкой считается sha256 файла и сохраняется в метаданные объекта, после загрузки размер и хеш сверяются.

```
PS C:\Users\ashai\project\inzvk\zad3\hh-skills-mapreduce> python src\upload.py
Хранилище: http://127.0.0.1:9000
Бакет hh-lake уже существует
Локальный файл: it_vacancies_2024.csv, 847,619,384 байт, sha256=303db922b541e444...
Загружено за 28.0 с (28.8 MiB/c)

Проверка объекта в хранилище (HEAD):
  s3://hh-lake/raw/it_vacancies_2024.csv
  ContentLength = 847,619,384 байт
  ETag          = "0e6d21890db664d9bf83e1427665b106-13"
  LastModified  = 2026-10-18 09:57:52+00:00
  sha256 (meta) = 303db922b541e444...
  Размер и хеш совпадают с локальным файлом: True

Содержимое бакета hh-lake:
    169,061,149  curated/it_vacancies_2024.parquet
    847,619,384  raw/it_vacancies_2024.csv
            174  results/analysis/combinatorics_by_skill_count.csv
         13,272  results/analysis/pair_uplift.csv
          1,027  results/analysis/regions_distinctive_skills.csv
            934  results/analysis/regions_key_metrics.csv
          1,258  results/analysis/regions_salary_by_skill.csv
          3,116  results/analysis/regions_skill_demand.csv
            166  results/analysis/salary_by_experience.csv
            118  results/analysis/salary_by_skill_count.csv
            767  results/analysis/skills_top_vs_bottom_quartile.csv
         31,538  results/analysis/triple_uplift.csv
         63,474  results/duckdb/k1_skills.csv
      1,619,186  results/duckdb/k2_pairs.csv
     15,411,648  results/duckdb/k3_triples.csv
            462  results/duckdb/summary.json
         63,474  results/duckdb_parquet/k1_skills.csv
      1,619,186  results/duckdb_parquet/k2_pairs.csv
     15,411,648  results/duckdb_parquet/k3_triples.csv
            475  results/duckdb_parquet/summary.json
         63,454  results/mapreduce/k1_skills.csv
      1,617,494  results/mapreduce/k2_pairs.csv
     15,391,178  results/mapreduce/k3_triples.csv
            714  results/mapreduce/summary.json
         63,454  results/mapreduce_failtest/k1_skills.csv
      1,617,494  results/mapreduce_failtest/k2_pairs.csv
     15,391,178  results/mapreduce_failtest/k3_triples.csv
            842  results/mapreduce_failtest/summary.json
         63,454  results/mapreduce_optimized/k1_skills.csv
      1,617,494  results/mapreduce_optimized/k2_pairs.csv
     15,391,178  results/mapreduce_optimized/k3_triples.csv
            724  results/mapreduce_optimized/summary.json
          1,066  results/profiling/runs.csv
            408  results/profiling/summary.csv
            414  results/profiling/sweep_workers.csv
```

Рис. 2. Загрузка: 847 619 384 байт за 28,0 с, размер и sha256 совпадают с локальным файлом; содержимое бакета

Полный датасет лежит в зоне `raw/`, и вариант из него отбирают оба способа подсчёта, каждый своим условием. Дополнительно сегмент варианта сохранён отдельным объектом: скрипт `export_segment.py` отбирает вакансии SQL-условием из раздела 3, записывает все 23 исходные колонки и проверяет, что по выгрузке получаются те же контрольные числа.

```
Сегмент «Программист, разработчик на Python» записан: s3://hh-lake/segment/variant2_python_developers.csv за 4.9 с
Проверка выгрузки: строк 7,545, уникальных vacancy_id 7,545, колонок 23
Повторный отбор варианта внутри выгрузки: 7,545 вакансий (условие идемпотентно)
Контрольные числа по выгрузке: с навыками 6,637; сумма частот k1 48,857, k2 223,836, k3 896,960
Совпадают с этапом 3 (7 545 / 6 637 / 48 857 / 223 836 / 896 960): True
```

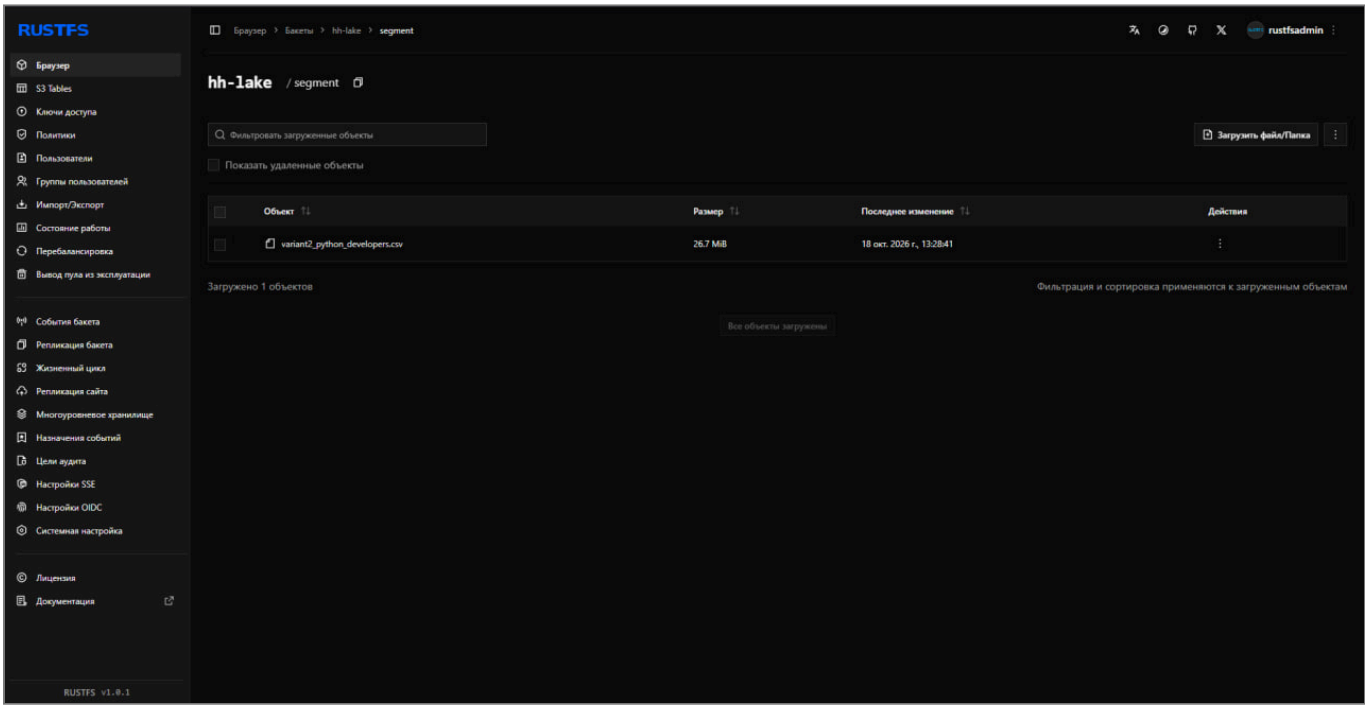


Рис. 3. Консоль RustFS: выгрузка сегмента в зоне `segment/`, 26,7 МиБ

3. Условие варианта

Вакансия относится к варианту 2, если её роль — «Программист, разработчик» и Python указан среди навыков или в названии вакансии. Навыки — уникальные непустые значения поля `key_skills`, разделённые «;», без пробелов по краям, в нижнем регистре.

Название учитывается потому, что поле навыков заполнено не всегда. По одним навыкам в вариант попадает 6 508 вакансий, ещё 1 037 содержат Python только в названии — например, «Python Developer» с пустым полем навыков. Другие роли (аналитики, DevOps со знанием Python) не учитываются: это чужие сегменты. Итого в варианте **7 545 вакансий, из них 6 637 с навыками**.

Условие реализовано дважды и независимо: на Python в MapReduce и на SQL в DuckDB.

```
def normalize_skills(raw):
    return sorted({s.strip().lower() for s in raw.split(";") if s.strip()})

def is_variant(role, name, skills):
    return role == VARIANT_ROLE and (VARIANT_KEYWORD in skills or VARIANT_KEYWORD in name.lower())
```

```
SELECT vacancy_id, skills
FROM (SELECT vacancy_id, role, name,
             list_sort(list_distinct(list_filter(
                 list_transform(string_split(coalesce(key_skills, ''), ';'), s -> lower(trim(s))),
                 s -> s <> ''))) AS skills
      FROM read_csv('s3://hh-lake/raw/it_vacancies_2024.csv', ...))
WHERE role = 'Программист, разработчик'
  AND (list_contains(skills, 'python') OR contains(lower(coalesce(name, '')), 'python'))
```

4. Собственный MapReduce

4.1. Устройство

Программа `mapreduce.py` читает объект напрямую из хранилища и проходит пять фаз:

- **split** — драйвер узнаёт размер объекта запросом HEAD и делит его на 26 диапазонов по 32 МиБ (последний меньше);
- **map** — каждый сплит обрабатывает отдельный процесс-маппер. Он читает свой диапазон байт запросом `GET` с заголовком `Range`, разбирает строки CSV, отбирает вакансии варианта и для каждой вакансии перебирает сочетания её навыков по 1, 2 и 3;
- **combine** — маппер суммирует сочетания у себя в памяти и отдаёт драйверу уже частоты, а не поток единиц;
- **shuffle** — ключи раскладываются по 4 партициям по остатку `crc32(ключ) % 4`, так что каждый ключ попадает ровно к одному редьюсеру;
- **reduce** — редьюсеры складывают частоты своих партиций от всех мапперов; результат для k = 1, 2, 3 записывается обратно в бакет.

Главная тонкость — границы сплитов. Граница диапазона почти всегда попадает в середину строки. Правило то же, что в Hadoop: строка принадлежит сплиту, в котором лежит её первый байт. Маппер читает диапазон с байта `start - 1`: если перед сплитом стоит перевод строки, первая строка своя, иначе её начало отбрасывается — она достанется предыдущему сплиту. Последнюю строку, вышедшую за границу, маппер дочитывает дополнительными запросами.

```
def read_split(s3, start, end, size):
    lo = start - 1 if start > 0 else 0
    data = _get_range(s3, lo, end - 1)
    if start > 0:
        nl = data.find(b"\n")
        if nl == -1:                        # весь сплит внутри чужой строки
            return b"", len(data)
        data = data[nl + 1:]
    if end < size and not data.endswith(b"\n"):
        ...                                 # дочитываем последнюю строку за границей сплита
```

Из-за перекрытий маппер читает чуть больше, чем объём объекта: 849 257 809 байт против 847 619 384, то есть 0,19 % сверху. Корректность разбиения проверяет тест `tests/test_splits.py`: на имитации хранилища каждая строка должна

попасть ровно в один сплит при размерах сплита от 1 байта до размера файла с запасом, с финальным переводом строки и без него. Все 20 конфигураций проходят, в каждой совпадают обычное и потоковое чтение.

4.2. Запуск и результаты

```
PS C:\Users\ashai\project\inzvk\zad3\hh-skills-mapreduce> python src\mapreduce.py
MapReduce | вариант 2: Программист, разработчик на Python
Источник: s3://hh-lake/raw/it_vacancies_2024.csv (847,619,384 байт)
Сплитов: 26 по 32 МиБ | мапперов: 6 | редьюсеров: 4

[map]
  завершено мапперов: 26/26 за 3.46 с; отказов: 0
  прочитано из S3: 849,257,809 байт; строк данных: 229,388; разобрано CSV: 229,388

[shuffle + reduce]
  редьюсер 0: ключей k1=784 k2=12,635 k3=89,146
  редьюсер 1: ключей k1=762 k2=12,706 k3=88,673
  редьюсер 2: ключей k1=749 k2=12,679 k3=88,999
  редьюсер 3: ключей k1=757 k2=12,645 k3=89,427
  shuffle+reduce: 0.52 с

[результат]
  вакансий варианта: 7,545; из них с навыками: 6,637
  k=1: уникальных      3,052 | сумма частот     48,857
  k=2: уникальных     50,665 | сумма частот    223,836
  k=3: уникальных    356,245 | сумма частот    896,960
  контрольный sha256: 5354f749b1a6f90370ab5df1c7cabf0d1d134c24a8e39b8490bd8eeefa9d98a8
  топ-3 k=1: python (6508), postgresql (2138), sql (2005)
  топ-3 k=2: postgresql + python (2097), python + sql (1988), git + python (1790)
  топ-3 k=3: git + postgresql + python (742), postgresql + python + sql (727), git + linux + python (702)

[запись результатов в хранилище]
  s3://hh-lake/results/mapreduce/k1_skills.csv          3,052 строк,       63,454 байт
  s3://hh-lake/results/mapreduce/k2_pairs.csv          50,665 строк,    1,617,494 байт
  s3://hh-lake/results/mapreduce/k3_triples.csv       356,245 строк,   15,391,178 байт
  s3://hh-lake/results/mapreduce/summary.json
  запись: 2.41 с

Итого: 7.62 с
```

Рис. 4. Запуск MapReduce: 26 сплитов, 6 мапперов, 4 редьюсера, результаты для k = 1, 2, 3 записаны в бакет

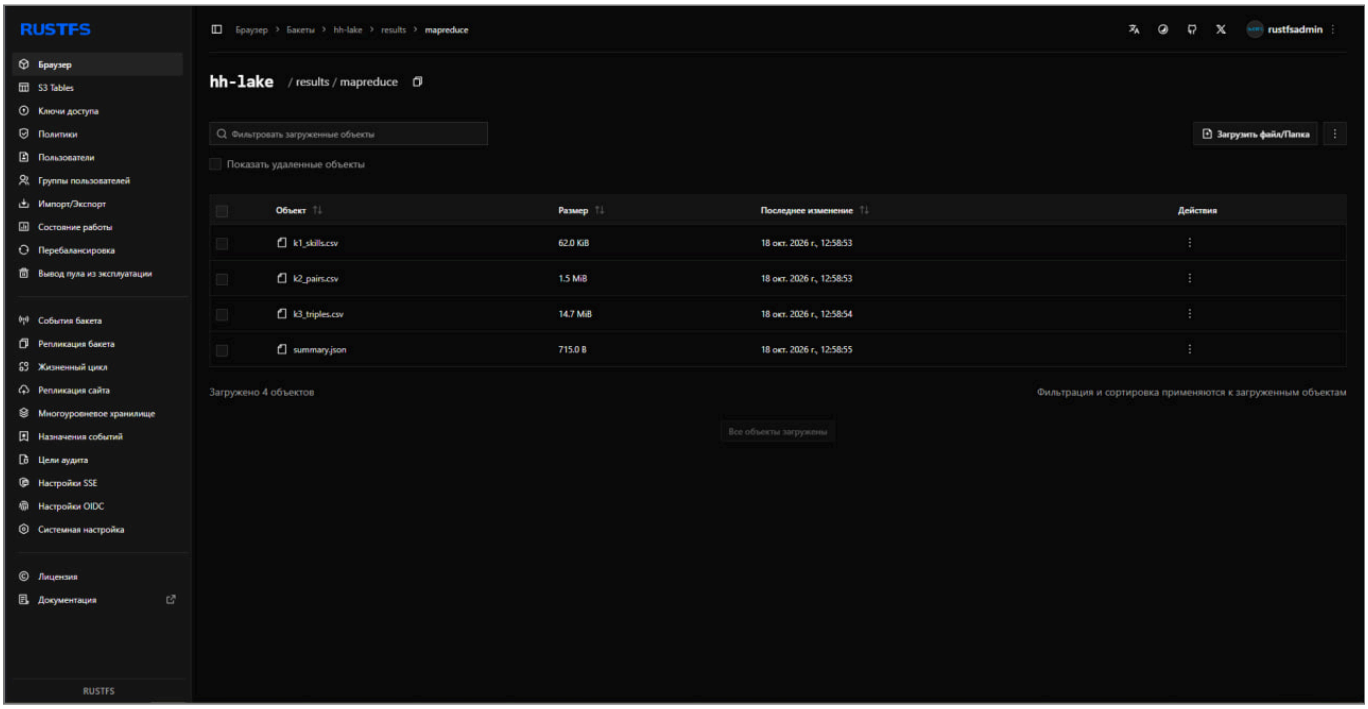


Рис. 5. Консоль RustFS: результаты MapReduce в `results/mapreduce/` — навыки, пары, тройки и сводка

Таблица 1. Самые частые навыки, пары и тройки (число вакансий)

Навыки	**Пары**	**Тройки**
python — 6 508	postgresql + python — 2 097	git + postgresql + python — 742
postgresql — 2 138	python + sql — 1 988	postgresql + python + sql — 727
sql — 2 005	git + python — 1 790	git + linux + python — 702
git — 1 828	linux + python — 1 592	git + python + sql — 653
linux — 1 619	docker + python — 1 244	docker + postgresql + python — 629
docker — 1 286	django framework + python — 915	docker + git + python — 533
django framework — 936	git + postgresql — 761	django framework + postgresql + python — 515
javascript — 747	javascript + python — 741	linux + postgresql + python — 498
c++ — 737	c++ + python — 734	linux + python + sql — 431
redis — 613	postgresql + sql — 734	docker + linux + python — 411

Типичный стек Python-разработчика по этим данным — Python, PostgreSQL и SQL, Git и Linux, Docker, из фреймворков — Django и FastAPI (612 вакансий).

4.3. Рост числа пар и троек

Таблица 2. Число сочетаний разного размера

	Уникальных	**Сумма частот**	**Встречаются один раз**
Навыки (k = 1)	3 052	48 857	1 284 (42 %)
Пары (k = 2)	50 665	223 836	25 158 (50 %)
Тройки (k = 3)	356 245	896 960	196 587 (55 %)

Рост объясняется числом сочетаний внутри одной вакансии. Вакансия с n навыками даёт $C(n, 2) = n(n − 1)/2$ пар и $C(n, 3) = n(n − 1)(n − 2)/6$ троек. В среднем у вакансии 7,36 навыка (медиана 6, максимум 30): вакансия с 7 навыками даёт 21 пару и 35 троек, с 15 — уже 105 пар и 455 троек, с 30 — 435 пар и 4 060 троек. Число пар растёт квадратично от длины списка навыков, троек — кубически.

Таблица 3. Вклад вакансий с разным числом навыков в общее число сочетаний

Навыков в вакансии	**Вакансий**	**Навыков (k = 1)**	**Пар (k = 2)**	**Троек (k = 3)**
1–3	987 (14,9 %)	2 372 (4,9 %)	1 930 (0,9 %)	545 (0,1 %)
4–6	2 475 (37,3 %)	12 300 (25,2 %)	25 171 (11,2 %)	27 184 (3,0 %)
7–10	1 967 (29,6 %)	16 282 (33,3 %)	60 410 (27,0 %)	132 537 (14,8 %)
11–15	860 (13,0 %)	10 728 (22,0 %)	62 347 (27,9 %)	224 228 (25,0 %)
16 и больше	348 (5,2 %)	7 175 (14,7 %)	73 978 (33,1 %)	512 466 (57,1 %)
Всего	**6 637**	**48 857**	**223 836**	**896 960**

Отсюда главный вывод: **5,2 % вакансий с 16 и более навыками дают 57,1 % всех троек**, хотя на их долю приходится лишь 14,7 % упоминаний навыков. Вакансии-«простыни» доминируют в тройках, поэтому частые тройки отражают в первую очередь именно их. Одновременно растёт разреженность: из $C(3\,052, 2) = 4\,655\,826$ возможных пар встречается только 50 665 (1,09 %), из 4,7 миллиарда возможных троек — 356 245, и больше половины из них — единичные.

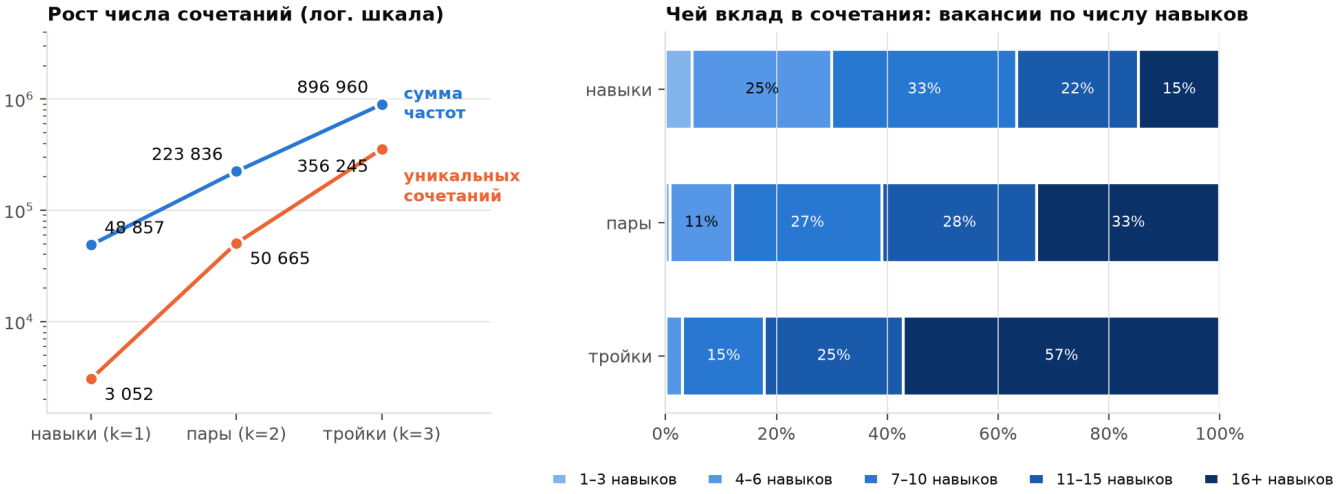


Рис. 6. Слева — рост числа сочетаний по размеру; справа — вклад вакансий с разным числом навыков

4.4. Отказ маппера

Устойчивость к отказам проверена в двух режимах. В режиме `crash` процесс-маппер завершается аварийно, не вернув результата, — это имитация падения узла. В режиме `exception` маппер бросает исключение. Драйвер в обоих случаях перезапускает только упавшую задачу, на место погибшего процесса поднимает новый, а результат неудачной попытки отбрасывает целиком — поэтому повтор не удваивает счётчики.

```
PS C:\Users\ashai\project\inzvk\zad3\hh-skills-mapreduce> python src\mapreduce.py --fail-map 3 --fail-mode crash --engine-name mapreduce_failtest
MapReduce | вариант 2: Программист, разработчик на Python
Источник: s3://hh-lake/raw/it_vacancies_2024.csv (847,619,384 байт)
Сплитов: 26 по 32 МиБ | мапперов: 6 | редьюсеров: 4
Инъекция отказа: мапперы [3], режим «crash»

[map]
  ОТКАЗ маппера #3, попытка 1: процесс pid=29712 аварийно завершился (код 17); результат попытки отброшен
  маппер #3 поставлен на повтор, попытка 2
  завершено мапперов: 26/26 за 3.44 с; отказов: 1
  прочитано из S3: 849,257,809 байт; строк данных: 229,388; разобрано CSV: 229,388

[shuffle + reduce]
  редьюсер 0: ключей k1=784 k2=12,635 k3=89,146
  редьюсер 1: ключей k1=762 k2=12,706 k3=88,673
  редьюсер 2: ключей k1=749 k2=12,679 k3=88,999
  редьюсер 3: ключей k1=757 k2=12,645 k3=89,427
  shuffle+reduce: 0.54 с

[результат]
  вакансий варианта: 7,545; из них с навыками: 6,637
  k=1: уникальных      3,052 | сумма частот     48,857
  k=2: уникальных     50,665 | сумма частот    223,836
  k=3: уникальных    356,245 | сумма частот    896,960
  контрольный sha256: 5354f749b1a6f90370ab5df1c7cabf0d1d134c24a8e39b8490bd8eeefa9d98a8
  топ-3 k=1: python (6508), postgresql (2138), sql (2005)
  топ-3 k=2: postgresql + python (2097), python + sql (1988), git + python (1790)
  топ-3 k=3: git + postgresql + python (742), postgresql + python + sql (727), git + linux + python (702)

[запись результатов в хранилище]
  s3://hh-lake/results/mapreduce_failtest/k1_skills.csv          3,052 строк,       63,454 байт
  s3://hh-lake/results/mapreduce_failtest/k2_pairs.csv          50,665 строк,    1,617,494 байт
  s3://hh-lake/results/mapreduce_failtest/k3_triples.csv       356,245 строк,   15,391,178 байт
  s3://hh-lake/results/mapreduce_failtest/summary.json
  запись: 2.48 с

Итого: 7.69 с
```

Рис. 7. Отказ маппера №3 в режиме crash: процесс завершился с кодом 17, задача повторена, итоговый sha256 совпадает со штатным прогоном

```
Инъекция отказа: мапперы [5, 17], режим «exception»
[map]
  ОТКАЗ маппера #5, попытка 1: RuntimeError('симулированный отказ маппера #5'); результат попытки отброшен
  маппер #5 поставлен на повтор, попытка 2
  ОТКАЗ маппера #17, попытка 1: RuntimeError('симулированный отказ маппера #17'); результат попытки отброшен
  маппер #17 поставлен на повтор, попытка 2
  завершено мапперов: 26/26 за 4.19 с; отказов: 2
  контрольный sha256: 5354f749b1a6f90370ab5df1c7cabf0d1d134c24a8e39b8490bd8eeefa9d98a8
```

Результат прогона с отказом записан в `results/mapreduce_failtest/` и сверен со штатным результатом скриптом `verify.py`: все 409 962 сочетания совпадают, расхождений 0.

5. Подсчёт в DuckDB

Скрипт `duckdb_counts.py` читает тот же объект `raw/it_vacancies_2024.csv` прямо из бакета через расширение httpfs, отбирает вакансии варианта SQL-условием из раздела 3 и строит таблицу пар «вакансия — навык». Пары и тройки навыков получаются самосоединением этой таблицы с условием `a.s < b.s < c.s`: так каждое сочетание берётся ровно один раз и в том же упорядоченном виде, что и в MapReduce. Результаты записываются обратно в бакет командой `COPY ... TO 's3://...'`.

```
SELECT a.s AS skill1, b.s AS skill2, c.s AS skill3, count(*) AS count
FROM edges a
JOIN edges b ON a.vacancy_id = b.vacancy_id AND a.s < b.s
JOIN edges c ON b.vacancy_id = c.vacancy_id AND b.s < c.s
GROUP BY ALL
```

Дополнительно DuckDB сам проверяет комбинаторику: сумма частот для каждого k сравнивается с суммой C(n, k) по вакансиям. Числа совпадают — 48 857, 223 836 и 896 960.

```
PS C:\Users\ashai\project\inzvk\zad3\hh-skills-mapreduce> python src\duckdb_counts.py
DuckDB 1.5.6 | вариант 2: Программист, разработчик на Python
Источник: 's3://hh-lake/raw/it_vacancies_2024.csv'
Потоков DuckDB: 12

[фильтр варианта] 1.61 с
  вакансий варианта: 7,545; из них с навыками: 6,637

[подсчёт]
  k=1: уникальных      3,052 | сумма частот     48,857 | ожидалось сочетаний     48,857 | 0.01 с
  k=2: уникальных     50,665 | сумма частот    223,836 | ожидалось сочетаний    223,836 | 0.04 с
  k=3: уникальных    356,245 | сумма частот    896,960 | ожидалось сочетаний    896,960 | 0.26 с

[топ-3]
  k=1: python (6508), postgresql (2138), sql (2005)
  k=2: postgresql + python (2097), python + sql (1988), git + python (1790)
  k=3: git + postgresql + python (742), postgresql + python + sql (727), git + linux + python (702)

[запись результатов в хранилище: COPY ... TO s3://]
  s3://hh-lake/results/duckdb/k1_skills.csv
  s3://hh-lake/results/duckdb/k2_pairs.csv
  s3://hh-lake/results/duckdb/k3_triples.csv
  s3://hh-lake/results/duckdb/summary.json

Итого: 4.20 с
PS C:\Users\ashai\project\inzvk\zad3\hh-skills-mapreduce>
```

Рис. 8. Запуск DuckDB: те же 7 545 вакансий и то же число сочетаний; результаты записаны в бакет

Файлы результатов DuckDB на несколько байт больше, чем у MapReduce, например `k1_skills.csv` — 63 474 байта против 63 454. Причина в оформлении CSV: DuckDB заключает в кавычки значения с символом «#» (`"c#"`), Python — нет. Таких навыков 10, по две кавычки на каждый — ровно 20 байт. После разбора CSV файлы совпадают построчно, включая порядок строк.

6. Сверка результатов

Скрипт `verify.py` скачивает из бакета результаты обоих способов, разбирает их в словари «сочетание → частота» и сравнивает по каждому ключу. Дополнительно для каждого результата независимо считается контрольная сумма: все строки результата сортируются, склеиваются в один текст, и от него берётся sha256. Совпадение хешей означает, что совпадает каждая из 409 962 строк до последней частоты.

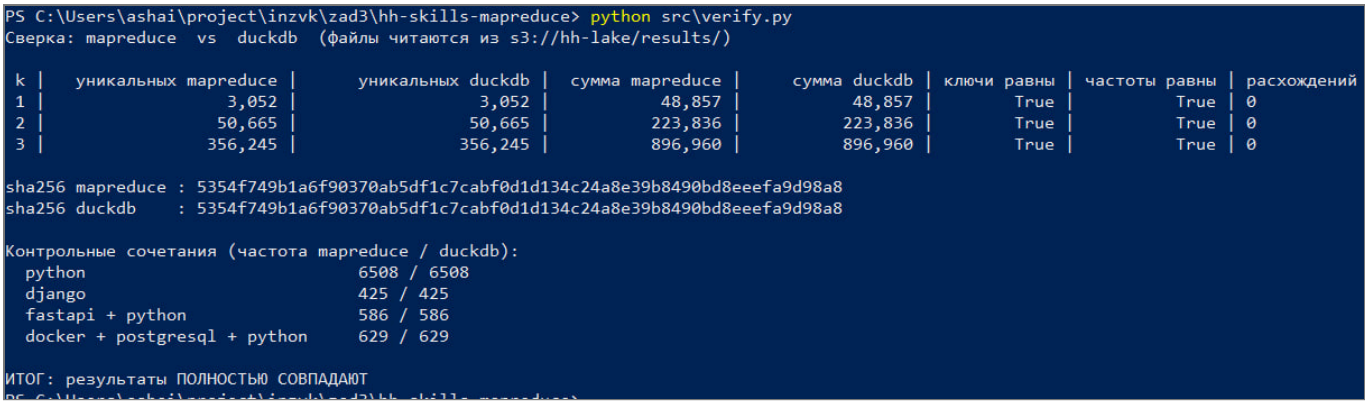


Рис. 9. Сверка MapReduce и DuckDB: ключи и частоты совпадают для k = 1, 2, 3, расхождений 0, хеши равны

Таблица 4. Все проведённые сверки

Что сравнивается	**Результат**
MapReduce и DuckDB	409 962 сочетания, 0 расхождений
MapReduce штатный и с отказом маппера	0 расхождений
Оптимизированный MapReduce и DuckDB по Parquet (бонус)	0 расхождений
Контрольные числа по выгрузке сегмента и этапа 3	совпадают
Таблицы этапов 5–6 и их пересчёт на чистом Python	878 строк, 0 расхождений

Контрольный sha256 всех прогонов: `5354f749b1a6f90370ab5df1c7cabf0d1d134c24a8e39b8490bd8eeefa9d98a8`.

7. Профилирование по времени и памяти

Скрипт `profile_runs.py` запускает каждую конфигурацию 5 раз отдельным процессом. Время измеряется от старта процесса до его завершения, включая запуск интерпретатора. Память — пик суммарного объёма занятой памяти (RSS) всего дерева процессов: у MapReduce это драйвер вместе со всеми мапперами и редьюсерами, иначе их память не попала бы в подсчёт. Объём опрашивается каждые 20 мс. Запись результатов в бакет при профилировании отключена, чтобы сравнивались только вычисления.

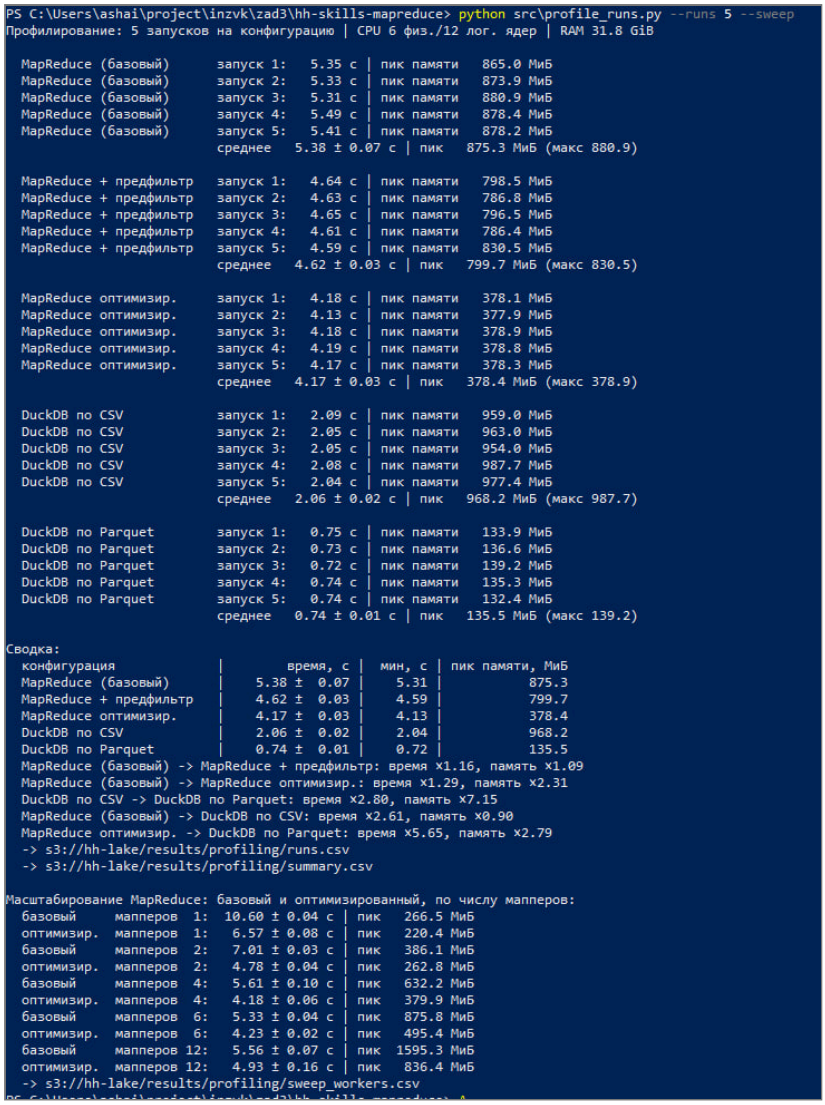


Рис. 10. Профилирование: по 5 запусков на конфигурацию и масштабирование MapReduce по числу мапперов

Таблица 5. Время и пик памяти, 5 запусков

Конфигурация	Время, с	Мин., с	Пик памяти, МиБ
MapReduce базовый	5,38 ± 0,07	5,31	875,3
MapReduce + предфильтр	4,62 ± 0,03	4,59	799,7
MapReduce оптимизированный	4,17 ± 0,03	4,13	378,4
DuckDB по CSV	2,06 ± 0,02	2,04	968,2
DuckDB по Parquet	0,74 ± 0,01	0,72	135,5

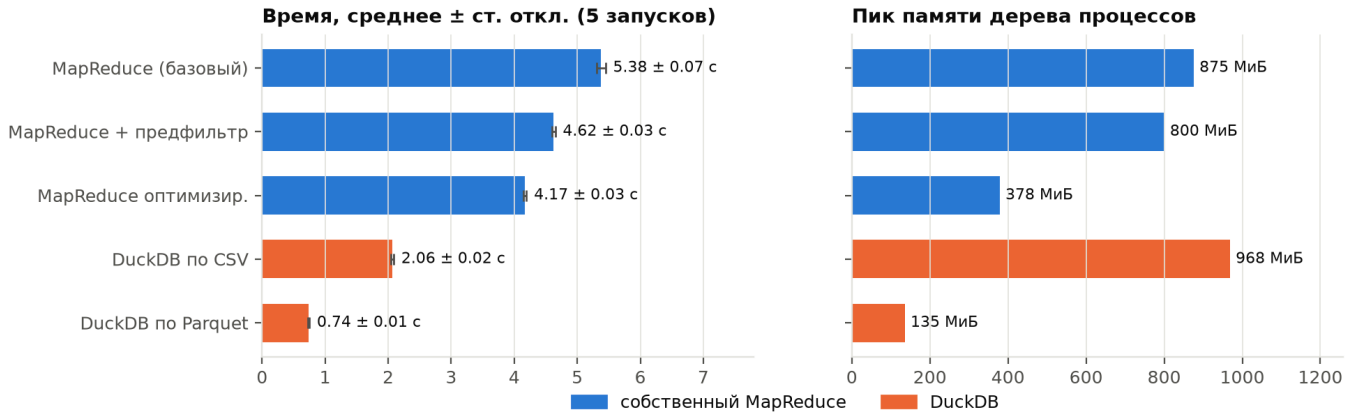


Рис. 11. Среднее время со стандартным отклонением и пик памяти по конфигурациям

Почему DuckDB быстрее в 2,6 раза. DuckDB — один процесс с движком на C++: он разбирает CSV параллельно в 12 потоков и обрабатывает данные векторами. Собственный MapReduce написан на Python, и часть его времени уходит на постоянные издержки. В Windows процессы-мапперы создаются заново, и каждый импортирует boto3. Каждая строка CSV разбирается интерпретируемым кодом. Результаты мапперов — около 450 тысяч ключей — сериализуются и передаются драйверу между процессами. В прогоне на рис. 4 map-фаза заняла 3,46 с из 7,62 с, остальное ушло на запуск процессов, shuffle и reduce и запись результатов в бакет.

Почему память почти одинакова при разной природе. У DuckDB по CSV пик 968 МиБ: движок держит буферы параллельного чтения 848-мегабайтного файла. У MapReduce память растёт с числом мапперов — каждый процесс держит интерпретатор с boto3 и свой 32-мегабайтный сплит, причём дважды: как байты и как список строк.

Таблица 6. Масштабирование MapReduce по числу мапперов, 3 запуска на точку

Мапперов	Базовый, с	Базовый, МиБ	Оптимизированный, с	Оптимизированный, МиБ
1	10,60 ± 0,04	266,5	6,57 ± 0,08	220,4
2	7,01 ± 0,03	386,1	4,78 ± 0,04	262,8
4	5,61 ± 0,10	632,2	4,18 ± 0,06	379,9
6	5,33 ± 0,04	875,8	4,23 ± 0,02	495,4
12	5,56 ± 0,07	1 595,3	4,93 ± 0,16	836,4

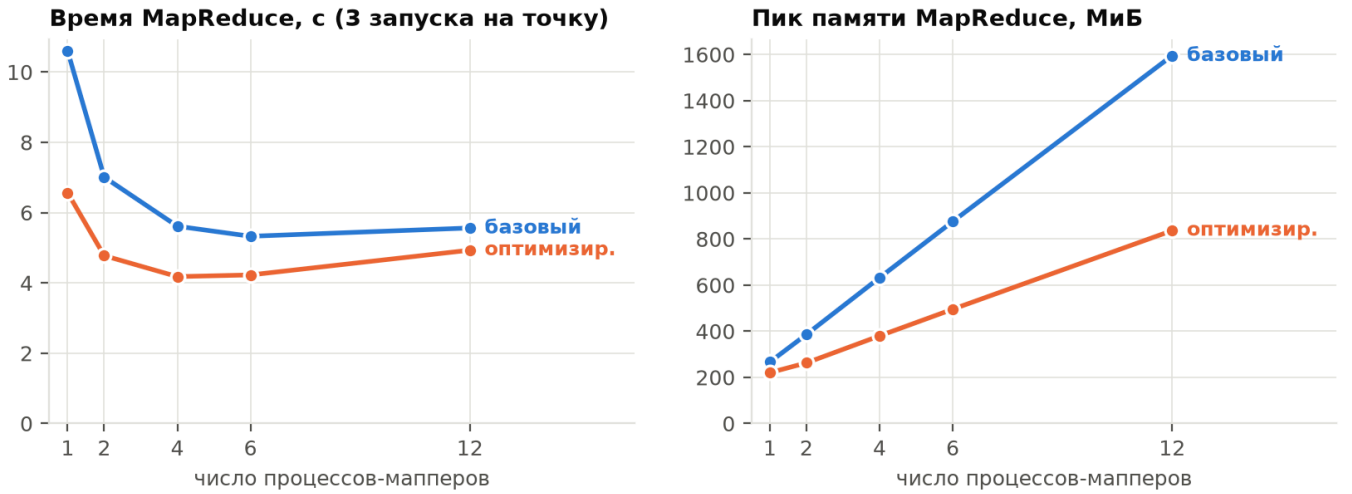


Рис. 12. Время и пик памяти MapReduce в зависимости от числа мапперов

Время перестаёт уменьшаться после 4–6 мапперов — на машине 6 физических ядер. На 12 мапперах время растёт: логические ядра делят физические, а запуск лишних процессов стоит дороже, чем даёт параллелизм. Память растёт линейно, примерно на 120 МиБ на каждого маппера в базовой версии и на 55 МиБ в оптимизированной. Поэтому оптимизированная версия запускается с 4 мапперами.

8. Бонус: оптимизация

Для MapReduce сделаны три оптимизации, каждая включается своим ключом, все вместе — ключом `--optimized`:

- **предфильтр** — строка вакансии варианта обязательно содержит байты названия роли и слово «python» в любом регистре. Строки без них отбрасываются до разбора CSV, и разбирается 13 342 строки вместо 229 388, в 17 раз меньше;
- **потоковое чтение** — сплит читается порциями по 1 МиБ и разбирается на лету, а не загружается в память целиком;
- **reduce в драйвере** — суммирование выполняется в процессе драйвера без второго пула процессов, который на Windows дорог в запуске.

Для DuckDB исходный CSV один раз переложен в колоночный формат Parquet в зоне `curated/`. Конвертация заняла 9,3 с, файл вышел 169 061 149 байт — 19,9 % от CSV. Запросу нужны 3 колонки из 23, и из Parquet читаются только они.

Таблица 7. Результат оптимизаций

Что сравнивается	Время, с	Память, МиБ	Выигрыш по времени	Выигрыш по памяти
MapReduce: базовый → оптимизированный	5,38 → 4,17	875 → 378	×1,29	×2,31
MapReduce на одном маппере	10,60 → 6,57	267 → 220	×1,61	×1,21
DuckDB: CSV → Parquet	2,06 → 0,74	968 → 136	×2,80	×7,15
Худший → лучший вариант	5,38 → 0,74	875 → 136	×7,3	×6,5

Предфильтр сильнее всего заметен, когда маппер один и упирается в процессор: время сокращается в 1,6 раза. При шести мапперах выигрыш меньше, потому что время уже определяется постоянными издержками — запуском процессов и чтением 848 МБ из хранилища. Потоковое чтение и reduce в драйвере почти не меняют время, но заметно снижают память: при шести мапперах пик падает с 800 до 495 МиБ. Для DuckDB переход на Parquet даёт больше всего: в 2,8 раза быстрее и в 7 раз меньше памяти. Оптимизации не изменили результат: оптимизированный MapReduce и DuckDB по Parquet дают тот же sha256, сверка показывает 0 расхождений.

9. Влияние навыков на зарплату

9.1. Подготовка зарплат

Зарплата берётся только в рублях. Точка зарплаты — середина вилки, если указаны обе границы, иначе единственная граница. Сумма до вычета налога пересчитывается на руки умножением на 0,87 (НДФЛ 13 % в 2024 году). Значения вне диапазона 20 000–1 000 000 ₽ отбрасываются: нижняя граница — около МРОТ 2024 года, ниже встречаются явные опечатки вроде 110 ₽. Остаётся 2 648 вакансий.

При первом расчёте у разных пар навыков медиана совпала до рубля — 261 000 ₽. Проверка показала причину: один работодатель разместил одну и ту же вакансию в девяти городах с одинаковой вилкой, и девять копий задавали медиану. Такие копии — **клоны**: вакансии одного работодателя с тем же названием, вилкой и набором навыков. Их оказалось 718 в 365 группах, при общем анализе зарплат каждая группа учитывается один раз. Остаётся **1 930 вакансий**, медиана на руки — **140 000 ₽** (первый квартиль 85 000, третий 200 100).

Таблица 8. Крупнейшие группы клонов и их влияние на медиану

Работодатель	Вакансия	Копий	Городов	З/п, ₽
id 68587	Middle Backend-разработчик (Python)	52	28	174 000
id 68587	Backend-разработчик Python	21	10	69 600
id 67611	Программист C++	17	4	147 900

Сочетание	Вакансий с клонами	Медиана, ₽	Без клонов	Медиана, ₽
python + clickhouse	97	261 000	66	250 000
python + airflow	42	261 000	32	287 500
sql + c++ + c#	36	43 500	22	55 450

9.2. Зарплата, опыт и число навыков

Требуемый опыт	Вакансий	Медиана, ₽
Нет опыта	238	52 200
1–3 года	906	110 000
3–6 лет	705	200 000
Более 6 лет	81	261 000

Навыков в вакансии	Вакансий	Медиана, ₽
0	180	162 540
1–3	211	150 000
4–6	586	140 000
7–10	531	150 000
11 и больше	422	114 050

Опыт — главный фактор: от «нет опыта» до «более 6 лет» медиана растёт в пять раз. Длина списка навыков, напротив, не признак квалификации: вакансии с 11 и более навыками платят на 19 % меньше медианы сегмента, а вакансии без заполненных навыков — даже больше среднего.

Таблица 9. Навыки, которые чаще встречаются в верхней или нижней четверти вакансий по зарплате

Навык	В верхних 25 %	В нижних 25 %
postgresql	42,2 %	21,1 %
docker	26,2 %	14,3 %
clickhouse	9,7 %	0,6 %
redis	13,7 %	4,9 %
kafka	9,3 %	0,9 %
fastapi	13,9 %	5,7 %
rabbitmq	10,4 %	2,6 %
kubernetes	7,9 %	1,3 %

Навык	В верхних 25 %	В нижних 25 %
javascript	8,4 %	27,2 %
html	4,2 %	22,8 %
css	1,9 %	14,5 %
mysql	4,9 %	15,3 %
java	5,6 %	15,1 %
php	3,5 %	13,0 %
c#	2,6 %	11,3 %
веб-программирование	1,6 %	10,2 %

9.3. Прибавка от пар и троек навыков

Прибавка пары считается относительно её **более частого** навыка: если навык A встречается чаще навыка B, то прибавка от добавления B равна разнице медиан вакансий с A и B и вакансий с A. Частоты навыков взяты из результата этапа 3 в бакете. Для тройки базой служит пара двух более частых навыков, а прибавка — эффект добавления третьего, самого редкого. Чтобы медианы были устойчивыми, учитываются пары не менее чем с 30 вакансиями с зарплатой (таких 237) и тройки не менее чем с 20 (таких 532).

Таблица 10. Десять пар навыков, сильнее всего повышающих, и десять — понижающих медиану (по Δ в рублях)

База	Медиана базы (n)	+ навык	Медиана пары (n)	Δ, ₽	Δ, %
python	139 600 (1 716)	airflow	287 500 (32)	+147 900	+105,9
python	139 600 (1 716)	dwh	261 000 (33)	+121 400	+87,0
sql	140 000 (539)	clickhouse	261 000 (32)	+121 000	+86,4
python	139 600 (1 716)	asyncio	250 000 (39)	+110 400	+79,1
python	139 600 (1 716)	clickhouse	250 000 (66)	+110 400	+79,1
postgresql	160 950 (572)	clickhouse	261 000 (51)	+100 050	+62,2
python	139 600 (1 716)	etl	239 625 (50)	+100 025	+71,7
sql	140 000 (539)	etl	240 000 (38)	+100 000	+71,4
postgresql	160 950 (572)	kafka	250 000 (42)	+89 050	+55,3
python	139 600 (1 716)	kafka	226 200 (59)	+86 600	+62,0
python	139 600 (1 716)	unity	50 000 (32)	−89 600	−64,2
sql	140 000 (539)	html	62 500 (74)	−77 500	−55,4
python	139 600 (1 716)	обучение и развитие	65 250 (37)	−74 350	−53,3
python	139 600 (1 716)	html5	74 500 (61)	−65 100	−46,6
python	139 600 (1 716)	веб-программирование	75 000 (88)	−64 600	−46,3
sql	140 000 (539)	css	76 125 (41)	−63 875	−45,6
postgresql	160 950 (572)	css	97 850 (30)	−63 100	−39,2
git	138 112 (576)	css	75 562 (52)	−62 550	−45,3
python	139 600 (1 716)	css	80 000 (125)	−59 600	−42,7
python	139 600 (1 716)	html	80 000 (204)	−59 600	−42,7

Таблица 11. Десять троек навыков, сильнее всего повышающих, и десять — понижающих медиану (по Δ в рублях)

База (пара)	Медиана базы (n)	+ навык	Медиана тройки (n)	Δ, ₽	Δ, %
postgresql + sql	160 950 (219)	airflow	304 500 (20)	+143 550	+89,2
python + postgresql	160 950 (558)	airflow	302 250 (22)	+141 300	+87,8
python + sql	140 000 (532)	airflow	275 000 (29)	+135 000	+96,4
python + sql	140 000 (532)	spark	261 000 (21)	+121 000	+86,4
python + sql	140 000 (532)	dwh	261 000 (25)	+121 000	+86,4
python + sql	140 000 (532)	clickhouse	261 000 (32)	+121 000	+86,4
python + git	135 000 (563)	clickhouse	239 250 (21)	+104 250	+77,2
python + sql	140 000 (532)	kafka	243 975 (24)	+103 975	+74,3
python + postgresql	160 950 (558)	clickhouse	261 000 (51)	+100 050	+62,2
postgresql + sql	160 950 (219)	clickhouse	261 000 (24)	+100 050	+62,2
python + sql	140 000 (532)	html	62 500 (74)	−77 500	−55,4
python + sql	140 000 (532)	html5	69 600 (27)	−70 400	−50,3
sql + git	134 850 (203)	html	67 500 (26)	−67 350	−49,9
python + sql	140 000 (532)	css	76 125 (41)	−63 875	−45,6
python + postgresql	160 950 (558)	css	97 850 (30)	−63 100	−39,2
python + git	135 000 (563)	html5	72 500 (22)	−62 500	−46,3
postgresql + sql	160 950 (219)	javascript	100 000 (36)	−60 950	−37,9
postgresql + sql	160 950 (219)	mysql	100 000 (29)	−60 950	−37,9
python + git	135 000 (563)	веб-программирование	75 000 (20)	−60 000	−44,4
python + sql	140 000 (532)	c++	80 475 (57)	−59 525	−42,5

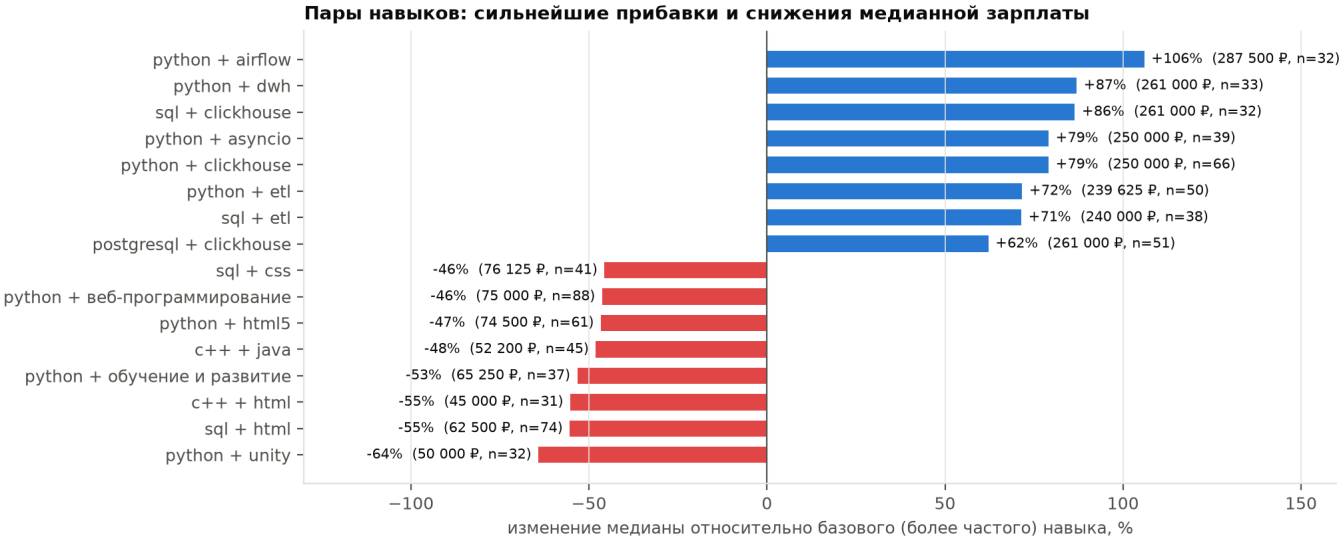


Рис. 13. Пары навыков с наибольшей прибавкой и наибольшим снижением медианы, в процентах к базовому навыку. Здесь пары отобраны по изменению в процентах, в таблице 10 — в рублях, поэтому наборы немного различаются

9.4. Итог: какие навыки повышают и понижают зарплату

Таблица 12. Итоговая таблица

Навыки	Эффект на медиану	Почему так
Повышают		
airflow, dwh, etl, spark, clickhouse	+62 … +106 %	Инженерия данных: построение хранилищ и конвейеров обработки. Это отдельная специализация с небольшим предложением специалистов, вакансии рассчитаны на опытных разработчиков.
asyncio, kafka	+55 … +79 %	Высоконагруженный бэкенд: асинхронность и очереди сообщений, требуются на уровне middle и выше. В верхней четверти по зарплате kafka встречается в 10 раз чаще, чем в нижней; туда же тянутся rabbitmq и redis (таблица 9).
Понижают		
unity, обучение и развитие	−53 … −64 %	Это не разработка, а преподавание: вакансии детских школ программирования, где «программист» ведёт курсы по Python и Unity. Медиана 50–65 тыс. ₽ — уровень зарплаты преподавателя.
html, html5, css, веб-программирование	−39 … −55 %	Вакансии «универсального» веб-разработчика: Python в них соседствует с вёрсткой. Обычно это начальные позиции в небольших компаниях; html встречается в нижней четверти по зарплате в 5 раз чаще, чем в верхней.
javascript, mysql, c++	−38 … −55 %	Длинные перечни языков без специализации характерны для стажировок и учебных позиций: пары c++ + java и c++ + html снижают медиану на 48 и 55 % (рис. 13), java и php заметно чаще встречаются в нижней четверти (таблица 9). Пример — 10 клонов вакансии «Программист-стажёр» с набором sql, c++, c#, java, php.

Важная оговорка: это связь, а не причина. Навык html не снижает зарплату сам по себе — он указывает на тип вакансии, где зарплата ниже. Медианы посчитаны по 20–200 вакансиям на сочетание, поэтому отдельные значения могут сдвинуться на новых данных, но направление эффекта устойчиво: те же навыки появляются и в парах, и в тройках, и в сравнении четвертей.

10. Сравнение регионов

Выбраны три региона с наибольшим числом вакансий варианта: Москва (3 636), Санкт-Петербург (972) и Екатеринбург (290). Регион вакансии — город её размещения, удалённые вакансии относятся к городу работодателя. Частоты навыков считаются по всем размещённым вакансиям, как на этапе 3. Клоны для регионов убираются только внутри города: копии одной вакансии в Москве и в Екатеринбурге — это две разные региональные вакансии.

Таблица 13. Ключевые показатели

Показатель	**Москва**	**Санкт-Петербург**	**Екатеринбург**
Вакансий варианта	3 636	972	290
Доля от всего сегмента	48,2 %	12,9 %	3,8 %
Вакансий с навыками	3 158	850	260
Медиана числа навыков	6	6	7
Вакансий с зарплатой, без клонов в городе	868	273	97
Доля вакансий с указанной зарплатой	27,1 %	34,1 %	41,4 %
Медиана зарплаты на руки, ₽	**191 400**	**150 000**	**110 000**
Первый квартиль, ₽	125 000	90 000	80 000
Третий квартиль, ₽	250 000	200 000	200 000
Медиана: нет опыта, ₽	80 000	60 000	67 625
Медиана: опыт 1–3 года, ₽	140 000	130 500	100 000
Медиана: опыт 3–6 лет, ₽	228 100	190 000	200 000
Доля удалённой работы	34,2 %	22,2 %	27,9 %
Доля вакансий без опыта	3,9 %	8,4 %	9,0 %
Доля вакансий с опытом более 6 лет	7,8 %	6,0 %	5,9 %

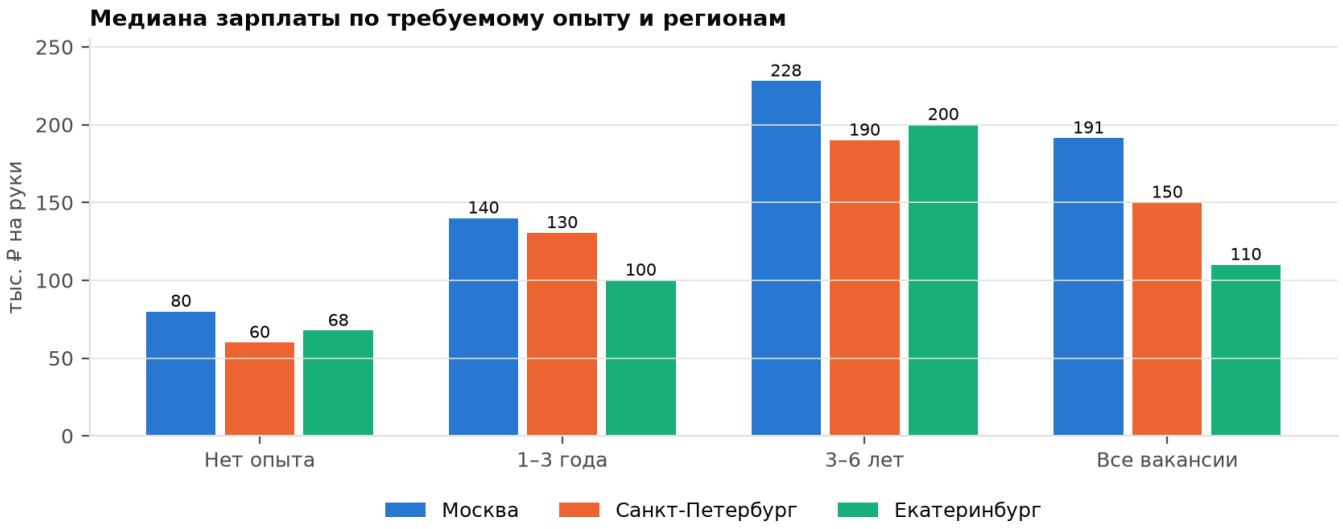


Рис. 14. Медиана зарплаты по требуемому опыту в трёх регионах

Таблица 14. Востребованность навыков, пар и троек (число вакансий и доля среди вакансий региона с навыками)

Москва	**Санкт-Петербург**	**Екатеринбург**
Навыки		
python — 3 087 (97,8 %)	python — 838 (98,6 %)	python — 252 (96,9 %)
postgresql — 1 027 (32,5 %)	postgresql — 292 (34,4 %)	postgresql — 93 (35,8 %)
sql — 933 (29,5 %)	git — 238 (28,0 %)	git — 85 (32,7 %)
git — 763 (24,2 %)	linux — 228 (26,8 %)	linux — 76 (29,2 %)
linux — 763 (24,2 %)	sql — 219 (25,8 %)	sql — 70 (26,9 %)
docker — 693 (21,9 %)	docker — 157 (18,5 %)	docker — 51 (19,6 %)
django framework — 395 (12,5 %)	django framework — 120 (14,1 %)	django framework — 44 (16,9 %)
fastapi — 340 (10,8 %)	c++ — 95 (11,2 %)	javascript — 36 (13,8 %)
Пары		
postgresql + python — 1 001 (31,7 %)	postgresql + python — 289 (34,0 %)	postgresql + python — 91 (35,0 %)
python + sql — 923 (29,2 %)	git + python — 233 (27,4 %)	git + python — 81 (31,2 %)
linux + python — 749 (23,7 %)	linux + python — 224 (26,4 %)	linux + python — 74 (28,5 %)
git + python — 738 (23,4 %)	python + sql — 216 (25,4 %)	python + sql — 69 (26,5 %)
docker + python — 660 (20,9 %)	docker + python — 154 (18,1 %)	docker + python — 51 (19,6 %)
Тройки		
git + linux + python — 326 (10,3 %)	git + postgresql + python — 102 (12,0 %)	git + postgresql + python — 37 (14,2 %)
postgresql + python + sql — 326 (10,3 %)	git + linux + python — 98 (11,5 %)	git + linux + python — 30 (11,5 %)
docker + postgresql + python — 319 (10,1 %)	docker + postgresql + python — 87 (10,2 %)	docker + git + python — 28 (10,8 %)
git + postgresql + python — 305 (9,7 %)	django framework + postgresql + python — 86 (10,1 %)	postgresql + python + sql — 28 (10,8 %)
git + python + sql — 263 (8,3 %)	postgresql + python + sql — 85 (10,0 %)	django framework + postgresql + python — 26 (10,0 %)

Таблица 15. Отличительные навыки региона: во сколько раз доля навыка выше, чем по всему сегменту (не менее 15 вакансий)

Регион	**Навыки**
Москва	arenadata ×1,9; postgres ×1,9; hive ×1,8; restful api ×1,8; pyspark ×1,7
Санкт-Петербург	elasticsearch ×2,2; django rest framework ×2,1; ci/cd ×1,8; unix ×1,5; qt ×1,5; pytest ×1,5
Екатеринбург	css ×2,2; golang ×2,0; базы данных ×1,6; redis ×1,5; html ×1,3; javascript ×1,2

Таблица 16. Медиана зарплаты на руки по ведущим навыкам сегмента, ₽ (в скобках — вакансий с зарплатой)

Навык	**Москва**	**Санкт-Петербург**	**Екатеринбург**
python	180 000 (769)	150 000 (240)	110 000 (83)
postgresql	200 000 (272)	177 500 (88)	200 000 (31)
sql	200 000 (228)	150 000 (58)	170 000 (29)
git	175 000 (239)	140 000 (87)	110 000 (37)
linux	176 675 (196)	150 000 (58)	90 000 (33)
docker	200 000 (188)	147 900 (45)	204 400 (14)
django framework	169 825 (118)	135 000 (50)	88 750 (12)
javascript	140 000 (125)	135 000 (31)	85 000 (16)
c++	174 000 (71)	92 500 (26)	— (9)
redis	200 000 (86)	190 000 (24)	174 000 (19)

Наблюдения

- **Москва — почти половина рынка и самые высокие зарплаты.** Медиана 191 400 ₽ выше петербургской на 28 % и екатеринбургской на 74 %.
- **Разрыв создают начинающие, а не опытные.** С опытом 3–6 лет Екатеринбург платит 200 000 ₽ — 88 % московского уровня, а с опытом 1–3 года — 100 000 ₽, лишь 71 %. Опытный разработчик в регионе получает почти столько же, сколько в столице; дешевле обходится именно младший уровень.
- **Регионы охотнее берут без опыта.** Доля таких вакансий 8,4 % в Петербурге и 9,0 % в Екатеринбурге против 3,9 % в Москве. Москва, наоборот, чаще ищет сеньоров: 7,8 % вакансий требуют опыта более 6 лет.
- **Низкая медиана Екатеринбурга — следствие структуры спроса.** Ключевые навыки бэкенда оплачиваются там на московском уровне: postgresql — 200 000 ₽, docker — 204 400 ₽. А django, linux и javascript — 85–90 тыс. ₽. Регион перекошен в сторону веба и начальных позиций, что и тянет общую медиану вниз. Выборки по отдельным навыкам здесь небольшие (12–37 вакансий), поэтому это скорее тенденция, чем точная оценка.
- **У каждого города свой профиль.** Москва выделяется инструментами больших данных: arenadata, hive, pyspark. Петербург — продуктовой веб-разработкой и инженерной культурой: elasticsearch, django rest framework, ci/cd, pytest, а также qt. Екатеринбург — вёрсткой и Go: css в 2,2 раза и golang в 2 раза чаще среднего.
- **Ядро стека одинаково везде.** Первая пара во всех трёх городах — postgresql + python, у трети вакансий; набор частых троек тоже совпадает: git, linux, docker, postgresql вокруг Python.
- **Зарплату чаще указывают в регионах:** 41,4 % вакансий в Екатеринбурге против 27,1 % в Москве. Удалённая работа, наоборот, чаще всего предлагается из Москвы — 34,2 % вакансий.

11. Находка о качестве данных

Часть навыков работодатели набирают в русской раскладке: например, «c#» с кириллической «с» вместо латинской «c». Для любой системы подсчёта это разные строки. Скрипт `data_quality.py` нашёл 18 таких групп-двойников:

Навык	Упоминаний	Двойник	Упоминаний	Прибавка при склейке
c#	316	«c#», кириллическая «с»	65	+21 %
c++	737	«c++», кириллическая «с»	13	+2 %
c/c++	418	«c/c++»	6	+1 %
computer vision	112	«computer vision»	4	+4 %
tcp/ip	101	«tcp/ip», три кириллические буквы	3	+3 %

Всего у двойников 124 упоминания из 48 857 (0,25 %), на итоги работы это не влияет. Но для отдельного навыка эффект заметен: у C# двойник отнимает пятую часть упоминаний. Двойники сознательно не склеиваются: правило нормализации должно быть одинаковым и простым в обеих реализациях подсчёта. Похожая проблема — синонимы: «django framework» (936) и «django» (425) обозначают одно и то же, но тоже считаются разными навыками.

12. Выводы

- Хранилище RustFS развёрнуто, датасет загружен в бакет `hh-lake` с проверкой хеша, сегмент варианта — 7 545 вакансий — сохранён отдельно.
- Собственный MapReduce читает объект из хранилища 26 сплитами через Range-запросы, корректно обрабатывает границы сплитов и переживает отказ маппера без искажения результата. Он насчитал 3 052 навыка, 50 665 пар и 356 245 троек; результаты для k = 1, 2, 3 лежат в бакете.
- DuckDB с независимо написанным SQL-условием дал те же числа: сверка 409 962 сочетаний — 0 расхождений, контрольные хеши равны.
- Рост числа пар и троек объясняется сочетаниями внутри вакансии: число пар растёт квадратично, троек — кубически от числа навыков, и 5,2 % вакансий с 16+ навыками дают 57 % всех троек.
- DuckDB быстрее собственного MapReduce в 2,6 раза при сопоставимой памяти; с Parquet — в 7,3 раза быстрее и в 6,5 раза экономнее. Оптимизации MapReduce сократили память в 2,3 раза.
- Зарплату сильнее всего повышают навыки инженерии данных (airflow, dwh, clickhouse, etl, spark) и высоконагруженного бэкенда (asyncio, kafka), понижают — вёрстка (html, css), преподавательские навыки (unity,

обучение) и перечни языков без специализации.

- Москва платит больше всех, но разрыв с регионами создаёт в основном младший уровень: опытные разработчики в Екатеринбурге получают 88 % московской медианы.

Приложение. Репозиторий

Исходный код: https://github.com/Kekendr/hh-skills-mapreduce. Порядок запуска описан в README.

Файл	**Назначение**
`src/config.py`	подключение к RustFS, бакет, условие варианта
`src/upload.py`, `src/export_segment.py`	этап 2: загрузка датасета и выгрузка сегмента
`src/mapreduce.py`	этап 3: собственный MapReduce с повтором упавших мапперов
`src/duckdb_counts.py`	этап 3: подсчёт на SQL в DuckDB
`src/verify.py`	этап 4: сверка результатов по файлам из бакета
`src/variant_data.py`, `src/salary.py`	этап 5: подготовка зарплат и анализ прибавок
`src/regions.py`	этап 6: сравнение регионов
`src/combinatorics.py`, `src/data_quality.py`	рост числа сочетаний; навыки-двойники
`src/profile_runs.py`, `src/to_parquet.py`, `src/charts.py`	профилирование, бонус с Parquet, графики
`tests/test_splits.py`	корректность разбиения объекта на сплиты
`tests/crosscheck_analysis.py`	независимая перепроверка таблиц этапов 5–6 на чистом Python